B U I L D P L A N V 1 . 1

Aegis Browser - Free Android Build Plan

Android-only, zero-budget CI on GitHub Actions - a targeted revision of the master blueprint that removes all build costs

> Scope: Aegis Browser for Android (arm64) only
>
> Budget: $0 per month - public-repo GitHub Actions + Cloudflare free tier
>
> Companion kit: aegis-free-android-v1.1.0.zip
>
> Version 1.1 - November 1, 2026

Table of Contents

Executive Summary....1
1 What Changes in This Revision....2
2 The Free Compute Reality....2
3 Route B (Recommended): Vanilla Chromium on GitHub Actions....3
3.1 Repository layout and pipeline....3
3.2 The low-RAM argument set....4
4 Route A (Alternative): Forking Cromite....4
5 Signing and Distribution at Zero Cost....5
6 Phase 3 on Android: Sync Integration for Free....6
6.1 What changes from the blueprint....6
6.2 What must be tested before shipping....6
7 Revised Timeline and Build Cadence....6
8 Risks and Mitigations....7
9 Quickstart Checklist....7

Note: this Table of Contents is generated via field codes. After editing the document, right-click the TOC and choose "Update Field" to refresh page numbers.

Executive Summary

You decided to concentrate Aegis Browser on a single deliverable - the Android APK - and to spend nothing. This document re-plans the project around that constraint and ships the tooling it requires. The one paid element of the master blueprint, per-build spot virtual machines, is replaced by GitHub Actions on a public repository: free, unlimited build minutes on standard 4-core, 16 GB runners, with the finished APK signed and published to a draft GitHub Release automatically. The delivered companion kit, aegis-free-android-v1.1.0.zip, contains the complete pipeline as a single workflow file, the low-RAM build arguments with their rationale, the keystore and branding scripts, and a Kotlin sync client whose crypto and protocol layers both pass their conformance suites - 16 generated vector checks and 13 live end-to-end steps against a running worker.

Free compute buys capacity, not speed: a full arm64 Chromium build takes roughly 4 to 7 hours on the free runner, there is no cache to resume from, and each job is capped at 6 hours. The plan absorbs this by design - builds are batched weekly, patches stay small and independent, and the pipeline is one click. The recommended route builds vanilla Chromium with your own patch series (the blueprint's original architecture); the alternative of forking Cromite was evaluated and verified to be incompatible with the free runners as shipped, so it is kept only as a future feature source. The Windows phase is deferred unchanged, and the already-delivered Cloudflare sync backend is untouched.

Sections 1 and 2 state exactly what changes and what the free tier allows. Sections 3 to 5 cover the recommended pipeline, the Cromite assessment, and zero-cost signing and distribution. Sections 6 to 9 re-plan the sync integration on Android, set an eight-week cadence, list the risk register with mitigations, and end with a quickstart checklist.

1 What Changes in This Revision

This document is a targeted revision of the Aegis Browser master blueprint, driven by two decisions: **the first deliverable is Android-only**, and **the whole project must run at zero cost**. The zero-cost rule removes the one paid element the blueprint relied on, namely per-build spot virtual machines for compiling Chromium. It does not touch the parts that are already free and already working: the aegis-sync backend on Cloudflare Workers + D1, its end-to-end encryption design, and its 55-test verification suite. Those remain exactly as delivered in version 1.0 of the blueprint.

The build moves from rented machines to **GitHub Actions on a public repository**, which is free in unlimited quantities for standard runners. The trade is money for time: a build that would take roughly 1 hour on an 8-vCPU spot VM takes roughly 4 to 7 hours on the free 4-core runner, and every build starts from a clean checkout because no cache large enough for Chromium fits in the free tier. The plan below is engineered around that reality rather than pretending it away: builds are batched, patches are kept small, and the pipeline is fully automated so a build is something you start and walk away from.

The Windows desktop build (Phase 4 of the blueprint) is **deferred, not cancelled**. Nothing about the Android-only pivot damages it: patches accumulate as plain diff files that apply to any Chromium checkout, and the sync client lands in shared, portable code. When you eventually want Aegis on Windows, the same patch series and the same paid spot-VM kit delivered earlier (aegis-build, about 1-3 dollars per build) can be reused. Table 1 summarizes the cost model after this revision.

Table 1: Monthly cost of every Aegis component after the free-tier pivot

Component	Blueprint v1.0	This revision	Notes
Chromium Android builds	$1-3 per spot-VM build	$0 (GitHub Actions, public repo)	Unlimited free minutes on standard 4-core/16 GB runners
Sync backend (Workers + D1)	$0	$0	Unchanged; Cloudflare free tier already covers personal use
APK storage and distribution	$0 (GitHub Releases)	$0 (GitHub Releases)	Draft releases, 2 GB per file, no billing for public repos
Code signing	$0 (own keystore)	$0 (own keystore)	One-time local keystore creation; no CA or Play re-signing needed

Component	Blueprint v1.0	This revision	Notes
Play Store publishing	Optional, $25 one-time	Not required	Sideload from GitHub Releases instead; revisit only for distribution
Windows desktop build	Deferred in v1.0 anyway	Deferred	Reuse aegis-build spot-VM kit when needed

One consequence deserves plain language: building for free on GitHub means your repository must be **public**, so your patches and branding are visible to anyone. For a personal hardening project that publishes no secrets (keys stay in GitHub Secrets, the sync worker URL is the only sensitive string, and it can live in a private configuration file that is git-ignored), this is acceptable. Section 8 lists it as a formal risk with mitigations.

2 The Free Compute Reality

Nothing has changed about why the 8 GB laptop cannot compile Chromium. A full link of the Android browser wants well over 16 GB of RAM by itself, the toolchain and checkout consume on the order of 50-60 GB of disk, and a first build on the laptop would take a day or more even if memory allowed it. The master blueprint therefore moved builds to the cloud; this revision selects the only cloud option that costs nothing: GitHub's hosted runners, in the configuration available to public repositories.

The hardware and quota facts below were verified against GitHub's current documentation in November 2026 and shape every design decision in this plan.

Table 2: GitHub-hosted runner facts relevant to Aegis (verified November 2026)

Resource	Free-tier value	Consequence for Aegis
Actions minutes, public repo	Unlimited (standard runners)	Long Chromium builds are affordable; run them weekly or on demand
Actions minutes, private repo	2,000 minutes per month	Less than one Chromium build; therefore the repo must be public
Runner hardware (Linux standard)	4 vCPU, 16 GB RAM, 14 GB SSD documented	RAM is adequate only with low-memory build flags; disk must be supplemented by freeing preinstalled packages and using the larger /mnt volume
Job wall-clock cap	6 hours (360 minutes)	A full arm64 build sits close to this ceiling; the workflow sets a 350-minute timeout and leans on retries
Build cache (actions/cache)	10 GB per repository	Far below a Chromium build tree (tens of GB); no incremental builds - every run is a full build

Resource	Free-tier value	Consequence for Aegis
Artifact storage	500 MB (free plan)	Sufficient for one APK; the workflow also attaches the APK to a draft GitHub Release (2 GB per file)
Release assets	2 GB per file	Free, permanent hosting for every signed APK

Two runner-image details matter and are handled in the workflow. First, the ubuntu-latest image label is migrating from Ubuntu 24.04 to 26.04 between October 19 and November 19, 2026, and Chromium's dependency script treats a brand-new distribution release as untested territory; the workflow therefore pins **ubuntu-24.04** explicitly. Second, GitHub also offers free **arm64 hosted runners** (ubuntu-24.04-arm) for public repositories; they are an interesting future experiment, but every public Chromium fork cross-compiles Android builds from x64 hosts, so the workflow stays on the battle-tested path.

The final economic fact is the one to internalize: **free buys compute, not iteration speed**. There is no shared cache to resume from, so each build pays the full price: roughly 30-60 minutes to fetch and sync the source, and roughly 3.5-6 hours to compile and link an arm64 browser. The strategy sections that follow are all downstream of this single constraint.

3 Route B (Recommended): Vanilla Chromium on GitHub Actions

Route B is the direct implementation of the blueprint's original architecture - a vanilla Chromium checkout hardened by your own patch series - with the build infrastructure swapped from spot VMs to the free pipeline delivered in this kit. You own every line; the patch series stays in plain, reviewable diff files; and the license story is Chromium's permissive BSD-style one. It is the recommended route despite the slower feedback loop, because it builds exactly the browser you specified and nothing else.

3.1 Repository layout and pipeline

The companion kit **aegis-free-android-v1.1.0.zip** is a ready-to-push repository skeleton. Its core is a single workflow file that runs the entire pipeline on demand:

```
.github/workflows/build-aegis-apk.yml   one-click CI pipeline (the heart of the kit)
aegis/ci/arm64.gn                       low-RAM build flags, each documented inline
aegis/patches/                          your .patch files, applied in order
aegis/branding/apply.sh                 branding hook (app label; extend as you go)
scripts/create-release-keystore.sh      one-time signing keystore creation
android-sync/AegisSyncClient.kt         Phase 3 draft: Kotlin sync client
README.md                               quickstart, troubleshooting table
```

Each workflow run performs seven stages. It resolves the Chromium version (your input, or the current Chrome stable fetched from Google's public version-history API - 156.0.8078.79 on the day this plan was written); fetches the source with git history stripped to save roughly 20 GB and checks out the exact release tag; syncs dependencies including the Android SDK and NDK, which Chromium's tooling downloads automatically; applies your patches and branding hook; configures the build with the low-RAM arguments described next; compiles the chrome_public_apk target for arm64; and finally zipaligns and signs the APK with your release keystore, uploading it to a draft GitHub Release. Signing keys never sit in the repository - they are injected from four repository Secrets, so even a public repo leaks nothing.

Table 3: Pipeline stages and planning estimates (free 4-core runner)

Stage	What happens	Estimated duration
Prepare runner	Free ~25 GB of preinstalled packages, add 16 GB swap, install JDK 17	3-5 min
Fetch source	depot_tools + fetch --no-history chromium + checkout of the release tag	30-45 min
Sync dependencies	gclient sync + runhooks; downloads Android SDK/NDK via CIPD	20-40 min
Patch and configure	apply aegis patches, branding hook, gn gen with arm64.gn	3-5 min
Compile and link	autoninja -C out/arm64 chrome_public_apk	3.5-6 h
Sign and publish	zipalign, apksigner with your keystore, draft release upload	2-4 min

3.2 The low-RAM argument set

The 16 GB runner cannot afford the link-time appetite of an official Chromium build, so the kit pins a deliberately lean argument set. Each flag exists for a reason and each is documented inline in aegis/ci/arm64.gn:

Table 4: gn arguments used by the free pipeline and why

Argument	Effect on the free build
is_official_build = false	Official builds enable ThinLTO and PGO; the final link alone can demand 20-30+ GB of RAM and add 1-2 hours. Unofficial builds are still fully optimized (-O2).
use_thin_lto = false	Removes ThinLTO from the link - the single largest memory spike in a Chromium build.
chrome_pgo_phase = 0	Skips Profile-Guided Optimization; saves time and complexity. (Cromite also disables PGO in its shipped arguments.)

Argument	Effect on the free build
symbol_level = 0, v8_symbol_level = 0	No native debug symbols: shrinks the build tree by tens of GB - the difference between fitting and not fitting the runner disk.
is_component_build = false	Release-style static linking; the correct mode for a distributable APK.
disable_fieldtrial_testing_config = true	No field-trial experiment configs; behavior stays predictable.
ffmpeg_branding = "Chrome", proprietary_codecs = true	H.264/AAC support as shipped by Chrome and every public fork.
target_os = "android", target_cpu = "arm64"	One architecture only. Adding arm32 or x86 would roughly double compile time against the 6-hour ceiling.

When you someday build on hardware with 32 GB or more, flip is_official_build to true and drop the ThinLTO and PGO exceptions for a smaller, faster binary; nothing else in the pipeline changes.

4 Route A (Alternative): Forking Cromite

Cromite is the actively maintained successor of Bromite: a Chromium fork for Android with ad blocking, fingerprinting resistance, and a long list of privacy patches, all developed in the open. On paper it is a shortcut for Phase 2 of the blueprint - a decade-equivalent of hardening work you would otherwise write yourself, maintained by someone else through every upstream rebase. A fair assessment needs the catches, and there are two.

The first catch is that **Cromite's release pipeline cannot be reused for free**. This was verified directly against its repository: the build workflow selects its runner from an input parameter with values reserved for the maintainer's own machines, executes inside a custom Docker image, mounts private storage paths, and sets a 1,440-minute timeout - all hallmarks of a self-hosted runner, and impossible on GitHub's 6-hour-capped free machines as shipped. Forking Cromite therefore does not give you a one-click free build; it gives you a large patch series plus build scripts that you would have to adapt to the hosted-runner constraint (restricting targets to arm64 and trimming arguments). That adaptation is real work, and it lands on top of an unfamiliar codebase.

The second catch is licensing and trust. Cromite is GPLv3, so any browser built from it must be GPLv3 too - irrelevant for a personal project, but a real fork in the road if you ever distribute differently. And inheriting patches means trusting them; part of Aegis's stated purpose is that you understand every non-vanilla line.

Table 5: Route A versus Route B

Dimension	Route A: Cromite fork	Route B: Vanilla + free CI kit
Time to first APK	Slower start: adapt their scripts to free runners, learn their tree	Fastest start: push the kit, click Run
Hardening features	Ad blocking, fingerprinting resistance and more, pre-built	You add patches incrementally (blueprint Phase 2 batches)
Fit to 6-hour free CI	Unproven; their full build is designed for a 24-hour self-hosted runner	Designed for it: arm64-only, no LTO/PGO, stripped symbols
License	GPLv3 (inherited)	Chromium BSD-style
Understanding and control	Lower: large inherited patch set	Total: every line is yours
Upstream maintenance	Upstream rebases handled by Cromite's team	You rebase with the standard Chromium tools

The verdict: **start with Route B**, which is ready today and matched to the free constraint, and keep Route A as a future feature source - its patch series can even be mined selectively (ad-blocker integration being the most valuable) once your own pipeline is stable. This preserves the blueprint's original architecture while leaving the shortcut open.

5 Signing and Distribution at Zero Cost

Distribution follows the same no-purchase rule as building. Android does not require a certificate authority: APKs are signed with a key you generate once, locally, with the kit's script (scripts/create-release-keystore.sh, which needs only a JDK via keytool). The script creates a 4,096-bit RSA key valid for thirty years, prints the base64 form for the ANDROID_KEYSTORE_BASE64 secret, and reminds you of the three password/alias secrets. Four GitHub Secrets in total give the pipeline signing capability without the key ever entering the repository or the build logs.

The workflow signs each APK with apksigner after zipalign, verifies the certificate chain, and attaches the artifact to a **draft GitHub Release** tagged by Chromium version and run number. You publish the release with one click; the APK is then permanently downloadable at no cost (2 GB per-file limit, no billing for public repositories). Installation on the phone is ordinary sideloading: open the release page on the device, download, allow installs from that browser, install. Because the same keystore signs every future build, updates install over the top of previous versions - the only irreversible mistake would be losing the keystore file, which is why the README tells you to back it up in a password manager before the first build.

Two upgrades stay optional and paid, deliberately outside this plan's critical path. The Google Play Store remains a 25-dollar one-time developer fee if you ever want managed

distribution, and it would re-sign uploads automatically so the workflow would not change. F-Droid inclusion is free but brings its own build-infrastructure requirements; it is not worth pursuing for a personal browser. Neither is needed for Aegis to be fully usable on your own devices today.

6 Phase 3 on Android: Sync Integration for Free

6.1 What changes from the blueprint

The master blueprint placed the sync client in a native C++ component (components/aegis_sync) compiled into the browser. That plan remains valid for desktop, but on Android the free-CI reality favors a **Kotlin implementation in the browser's Java/Kotlin layer**: it compiles as part of the ordinary APK build (no extra build cost), it can call Chromium's UI machinery (settings pages, menu items) directly, and it is far easier to iterate on than native code when every test cycle costs hours. The delivered kit therefore includes android-sync/AegisSyncClient.kt - a faithful Kotlin port of the tested reference client aegis-client.mjs - alongside a vector-tested crypto core (AegisCrypto.kt).

The crypto hierarchy is unchanged and is reproduced exactly in the port: PBKDF2-SHA256 with 600,000 iterations derives the master key from the password; HKDF-SHA256 splits it into an authentication key and a key-encryption key; a 100,000-iteration PBKDF2 over the authentication key produces the login verifier; and a device-generated 32-byte vault key, wrapped with AES-256-GCM, encrypts every bookmark, tab, setting, password and autofill item with the item type, key and version bound as authenticated data. The port uses Android's built-in Conscrypt provider (javax.crypto) plus hand-written RFC 2898 PBKDF2 and RFC 5869 HKDF over raw bytes, because Android ships no public HKDF API and javax’s char[]-based PBKDF2 does not reproduce WebCrypto’s raw-byte password semantics (rationale documented in the AegisCrypto.kt header); the conformance suite in android-sync/test/ proves byte-exact agreement with the JS reference On current hardware the full derivation chain takes on the order of a second at login, which is imperceptible in a settings screen.

6.2 What must be tested before shipping

The crypto half of that port is now proven, not drafted. Kit v1.1 splits it into a pure-JVM AegisCrypto.kt with a zero-dependency conformance suite: sixteen generated vectors, each cross-checked at generation time against WebCrypto, Node’s native PBKDF2, and a literal mirror of the Kotlin algorithm - all sixteen pass, including a non-ASCII-password case and the full 600,000-iteration production chain. The suite also caught a real bug before it could ship: javax’s char[]-based PBKDF2 converts its input via UTF-8, so the Android client would have derived a different login verifier than the JS reference and cross-device sign-in would have failed; the fixed implementation derives PBKDF2 over raw bytes exactly as

WebCrypto does. Phase 3 therefore starts where the vectors end: the kit now also ships a live end-to-end driver (android-sync/test/E2EDriver.kt), and its full sequence - register, log in from two virtual devices, push bookmarks from one, pull and decrypt on the other, rotate the password, and confirm the vault key survives - already passes 13 of 13 steps against a locally-run worker (wrangler dev); run it once against your deployed worker as the final smoke test. Only after that does UI integration begin - a Sync screen in settings, a device list with revocation, and menu hooks - each delivered as a patch file in aegis/patches/ so it flows through the same free pipeline as everything else.

7 Revised Timeline and Build Cadence

The free pipeline changes the shape of the schedule: instead of fast, paid iteration, progress comes in weekly waves, each wave = one to two builds plus offline work (writing patches, editing branding, porting tests). The timeline below assumes one public repository, the delivered kit, and a few evenings a week.

Table 6: Eight-week plan on the free pipeline

Week	Milestone
Week 1	Repository created, kit pushed, keystore created, four Secrets set; first full build launched. Debug any first-run CI friction (Section 8 anticipates the likely ones).
Week 2	First signed APK installed on the phone via draft release; branding hook extended (icons, about strings); second build confirms repeatability.
Weeks 3-4	Blueprint patch batches 1-3 delivered one patch file at a time, each validated by a build: default search engine, Google URL tracker removal, baseline privacy flags.
Weeks 5-6	Phase 3 sync: JUnit port of the crypto test vectors, then the end-to-end sequence against your worker; Sync settings screen added as a patch.
Weeks 7-8	Passwords and autofill items flowing through the same client; device list and revocation UI; adopt the next-latest Chromium stable tag and confirm a clean rebase - the maintenance rhythm you will keep.

From week 8 onward the cadence is yours to choose: a monthly build tracking Chrome stable is a sensible rhythm (each Chromium stable release is a natural, tagged build input), with extra runs whenever a patch changes. If a faster loop is ever needed - for example while debugging a subtle native-code patch - the previously delivered aegis-build spot-VM kit remains the escape hatch at about 1-3 dollars per build, with every artifact and script identical in shape to what the free pipeline produces.

8 Risks and Mitigations

Every plan of this shape has known failure modes. They are listed here with likelihood and countermeasures so that a red build does not turn into a blocked project. The unifying

principle: the pipeline is free, so almost every failure's cheapest cure is simply to run it again after a small, targeted fix.

Table 7: Risk register for the free build pipeline

Risk	Likelihood	Impact	Mitigation
Build exceeds the 6-hour job cap and is killed	Medium	Lost run; retry same day	Workflow already sets 350-min timeout with arm64-only, no-LTO, zero-symbol args; keep them. Re-run is free. Persistent overruns: prune non-essential patches or move that batch to the spot-VM kit.
Runner runs out of disk	Low-Medium	Lost run	Kit frees ~25 GB, adds swap, works on /mnt, and strips symbols; do not add architectures or re-enable symbols.
First-run CI friction (depot_tools, hook, or dependency script hiccups)	Medium	One-time delay of hours	Pin ubuntu-24.04 (done), fix forward iteratively; the README troubleshooting table maps symptoms to causes.
Patch no longer applies after a Chromium upgrade	Medium (over months)	Blocked tag upgrade	Keep patches small and independent (kit rules); rebase with git apply --3way; drop or rewrite stale batches rather than accumulating rot.
Public repository exposes your work	Certain (by design)	Loss of privacy of the code, not of data	Secrets stay in GitHub Secrets; keystore never committed; sync server URL kept in a git-ignored config. Accept for a personal project or switch to the paid path.
Draft Kotlin client has porting bugs	Possible until tested	Sync errors, none cryptographic (server rejects bad verifiers)	Phase 3 gate: JUnit test vectors plus the two-device end-to-end sequence before any UI ships.

Two non-risks deserve a sentence so they stop consuming attention. Upstream Chromium stability is handled by pinning release tags - the workflow never builds the moving development branch. And Cloudflare free-tier exhaustion is not a realistic scenario for a personal sync service: the backend's request volume sits orders of magnitude below the free limits, exactly as analyzed in the master blueprint.

9 Quickstart Checklist

Everything below is free and takes under an hour of hands-on time; the first build then runs unattended for hours. The numbered steps match the kit README, which also carries the troubleshooting table.

1. Create a public GitHub repository (for example aegis-browser-android) and copy the contents of aegis-free-android-v1.1.0.zip into it, keeping the folder layout including the hidden .github directory.
2. Install any JDK locally and run scripts/create-release-keystore.sh with a strong password; store the keystore file in your password manager.
3. Add the four repository Secrets: ANDROID_KEYSTORE_BASE64, ANDROID_KEYSTORE_PASSWORD, ANDROID_KEY_ALIAS (aegis), ANDROID_KEY_PASSWORD.
4. Open the Actions tab, choose Build Aegis Browser APK, leave chromium_version empty for the latest Chrome stable, and run the workflow.
5. Wait 4-7 hours; open the resulting draft release and publish it.
6. Download the APK on your phone, allow installs from that browser, and install Aegis.
7. Deploy the sync backend if you have not yet (the five commands in the blueprint Appendix D), then verify a manual register/login from the reference client.
8. Start Phase 2: write the first small patch (default search engine), drop it into aegis/patches/, and run the pipeline again.

With that eighth step the project returns to its steady rhythm: small auditable patches in, signed APKs out, sync data flowing end-to-end encrypted through infrastructure you own - at a monthly bill of zero. The full blueprint remains the reference for architecture and the threat model; this document supersedes only its Phase 1 build strategy.